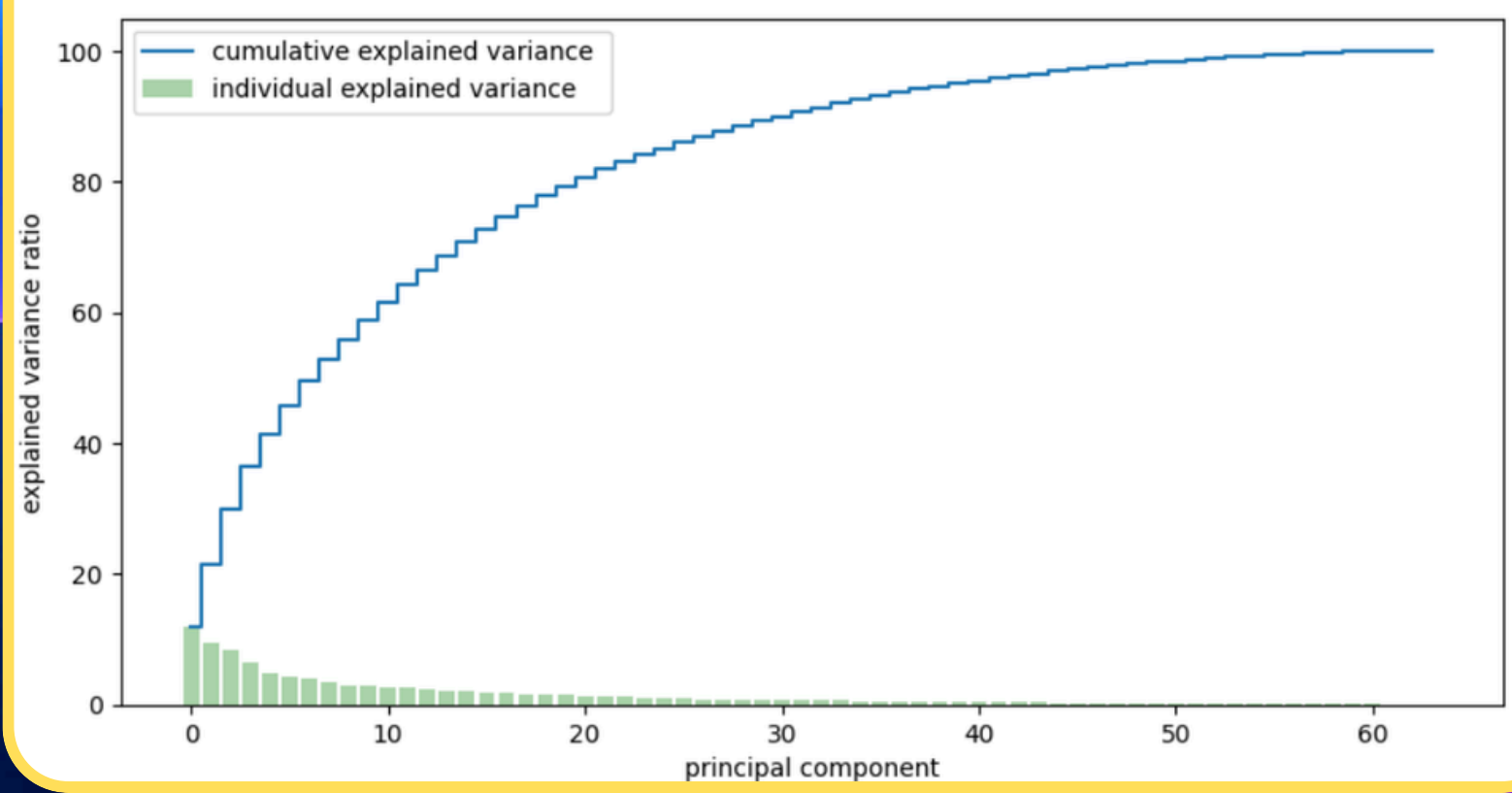
DATA SCIENCE

MACHINE LEARNING

PCA Hands-on Project

```
plt.figure(figsize=(10,5))
plt.bar(range(len(var_exp)),var_exp , alpha=0.333 , align='center' ,label='individual explained variance',color='green')
plt.step(range(len(cum_var_exp)) , cum_var_exp , where='mid' , label='cumulative explained variance ')
plt.ylabel('explained variance ratio')
plt.xlabel('principal component')
plt.legend(loc='best')
```

```
<matplotlib.legend.Legend at 0x171403b6120>
```



From Theory to Real Implementation — Master Principal Component Analysis with hands-on using real datasets.

ARYAN SHARMA
ASPIRING DATA SCIENTIST
MACHINE LEARNING ENTHUSIAST

THE PROBLEM

Handling High-Dimensional Data

Real-world datasets often have too many features, making analysis complex and computationally expensive. We explore this challenge using three classic datasets: Iris, Digits, and Breast Cancer.

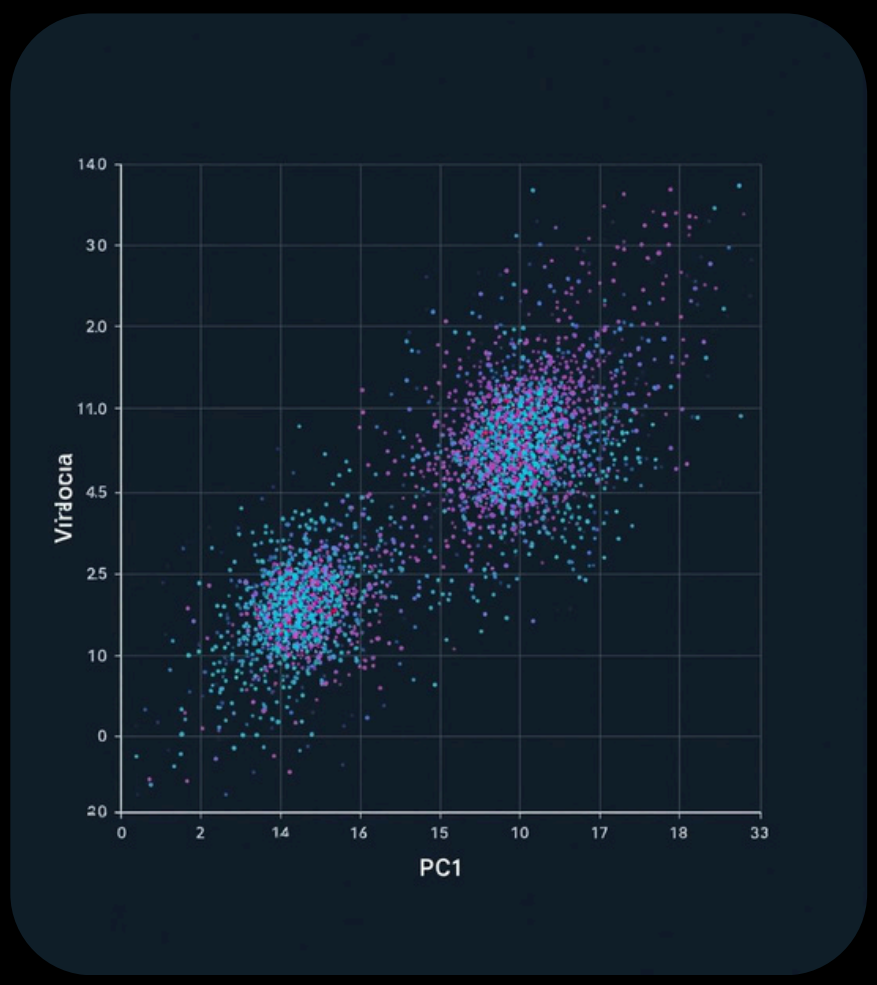
```
# the data is in  dictionary format so we have to convert
df=pd.DataFrame(data.data , columns=data.feature_names)
df
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	0.2419	0.07871	...	25.380	17.33	184.60	2019.0	0.16220
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	0.1812	0.05667	...	24.990	23.41	158.80	1956.0	0.12380
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	0.2069	0.05999	...	23.570	25.53	152.50	1709.0	0.14440
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	0.2597	0.09744	...	14.910	26.50	98.87	567.7	0.20980
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	0.1809	0.05883	...	22.540	16.67	152.20	1575.0	0.13740
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	0.1726	0.05623	...	25.450	26.40	166.10	2027.0	0.14100
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	0.1752	0.05533	...	23.690	38.25	155.00	1731.0	0.11660
566	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	0.1590	0.05648	...	18.980	34.12	126.70	1124.0	0.11390
567	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	0.2397	0.07016	...	25.740	39.42	184.60	1821.0	0.16500
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	0.1587	0.05884	...	9.456	30.37	59.16	268.6	0.08996

569 rows × 30 columns

IRIS DATASET

PCA RESULTS




```python
from sklearn.decomposition import PCA
```

```python
pca=PCA(n_components=20)
x_train_pca=pca.fit_transform(x_train)
```

```python
x_train_pca.shape
```

```
(455, 20)
```

Reduced 30 Features to 20 Principal Components

The Breast cancer dataset's 30 dimensions compressed into 20 PCs while preserving 96.49% of variance.

THE 4-STEP PIPELINE

PCA Steps Explained

PCA HANDSON

step 1 : Standardization

step 2 : Co-variance matrix - negative , positive or zero

step 3 : Compute eigen values and eigen vectors

```
the change in direction after the matrix transformation
or multiplication.(no change for doing transformation with identity matrix

the change of certain values is eigen values(no change for the same above
- how much the image is stretched or shrink is the eigen value.
```

step 4 : sort eigen values

pca_1= high variance

pca_2= low variance

1. Standardization
2. Covariance Matrix
3. Eigenvalues & Eigenvectors
4. Sort Components

Transform data through mathematical decomposition.

```python
from sklearn.decomposition import PCA
```

```python
pca=PCA(n_components=20)
x_train_pca=pca.fit_transform(x_train)
```

```python
x_train_pca.shape
```

```
(455, 20)
```

Applied PCA Before Classification

8x8 pixel images (64 dimensions) reduced to principal components while preserving digit patterns for accurate classification.

MODEL
COMPARISON

With PCA vs Without PCA

Compare accuracy and performance metrics to see how dimensionality reduction impacts model efficiency without sacrificing predictive power.

96.4%

With PCA

```
accuracy_score(y_test,y_pred_1binary)
```

```
0.9649122807017544
```

95.6%

Without PCA

```
from sklearn.metrics import accuracy_score, classification_report
```

```
accuracy_score(y_test,y_pred_binary)
```

```
0.956140350877193
```

INSIGHTS & TAKEAWAYS

Key Learnings from PCA

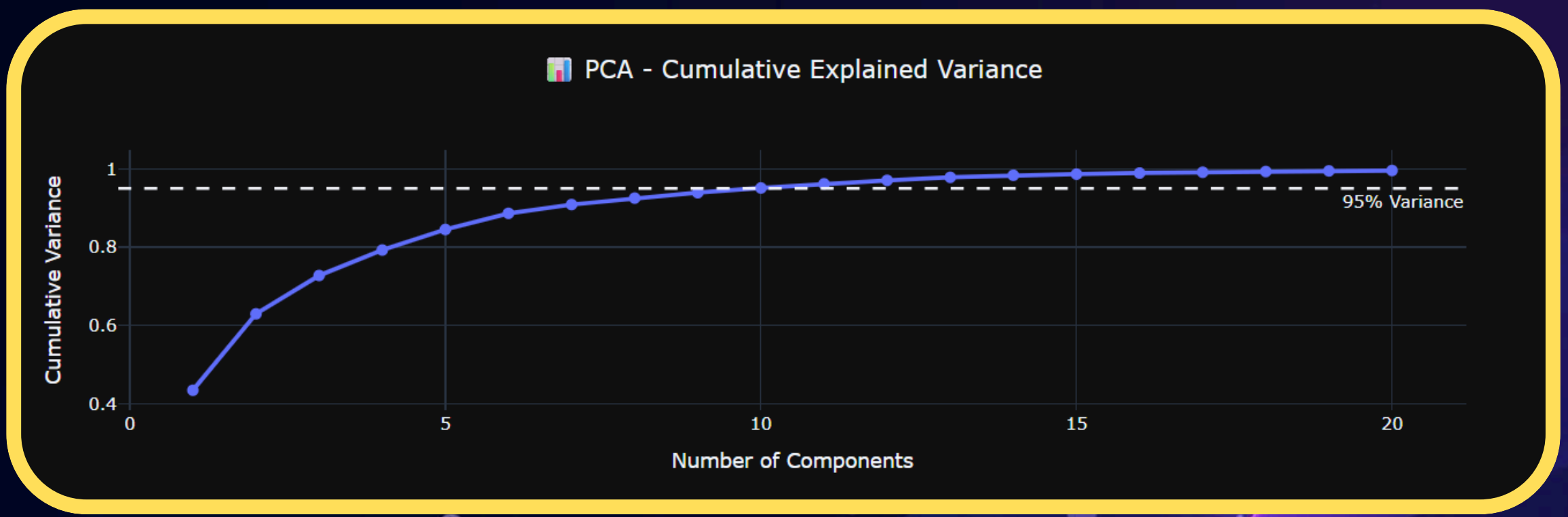
```python
from sklearn.preprocessing import StandardScaler

sc=StandardScaler()

x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)
```

- Standardization is crucial before applying PCA
- PCA captures maximum variance in fewer dimensions
- Reduces dimensionality with minimal information loss

NEXT SLIDE



Thank You!

*

LETS GRIND TOGETHER